**UNIVERSIDAD NACIONAL DE COSTA RICA**

**FACULTAD DE CIENCIAS EXACTAS Y NATURALES ESCUELA DE INFORMÁTICA**

**ARQUITECTURA DE COMPUTADORAS (EIF205)**

# REGISTRO DE EVIDENCIAS DEL PROYECYO MAZE8086

Integrantes del grupo:

**Belén Sandoval Méndez**

**Isaac Altamirano Pérez**

**Juan José Fernández Campos**

Profesor:

**César Valverde Canossa**

Fecha de entrega: **08-nov-2026**

## 1. Introducción

Maze8086 es un juego de laberinto desarrollado en lenguaje ensamblador Intel 8086. El jugador debe desplazarse por un mapa, recoger cinco monedas y encontrar la salida, mientras evita trampas que reducen las vidas disponibles. El proyecto utiliza una interfaz en modo texto, entrada por teclado y un marcador que presenta la puntuación, las vidas, los movimientos y el récord de la sesión.

Este documento reúne las evidencias de funcionamiento previstas para la versión integrada del juego. Se presentan en el mismo orden en que pueden comprobarse durante una partida, con una explicación breve del comportamiento que corresponde mostrar en cada imagen. Las capturas se incorporarán después de ejecutar cada situación en DOSBox-X.

## 2. Entorno utilizado

El código fuente está organizado en maze.asm, interfaz.inc, movimiento.inc y juego.inc. El archivo principal contiene el mapa y el control general de la partida; los archivos complementarios reúnen las rutinas de interfaz, desplazamiento y reglas. Se utiliza EMU8086 para compilar el programa y DOSBox-X para ejecutar el archivo MAZE.COM.

El juego permite desplazarse mediante las teclas WASD o las flechas. El mapa contiene paredes (#), monedas (C), trampas (X) y una salida (S). Las monedas suman diez puntos cada una y la salida concede cincuenta puntos adicionales cuando ya se recogieron todas, para un máximo de cien puntos por victoria.

## 3. Evidencias del funcionamiento

Las imágenes se presentan como figuras numeradas para mantener una relación clara entre las acciones del jugador y las reglas implementadas. En las situaciones donde no hay un cambio visible inmediato —como una colisión contra pared o una moneda ya recogida— se utiliza como referencia la captura anterior para comparar posición y marcador.

## ARQUITECTURA DEL SOFTWARE

### Flujo de ejecución

Maze8086 integra la lógica del laberinto, la entrada por teclado y el dibujo en modo texto en un programa DOS de 16 bits. El código separa el mapa y las reglas de la presentación. Cambiar la apariencia de una celda deja intacto su significado para las colisiones.

### Arranque y ciclo de juego

DOS carga el programa COM a partir del desplazamiento 0100h. La instrucción inicial salta a **inicio**, después de los datos y las rutinas incluidas. El programa copia CS a DS, configura el video y muestra el menú.

Durante la partida, INT 16h espera una tecla. El programa calcula la posición candidata, valida el destino, aplica las reglas y consulta el estado resultante antes de dibujar. Cada tecla de movimiento intenta avanzar una celda. La espera de teclado evita bucles de retraso y el juego carece de un reloj de simulación.

| Valor | Estado | Transiciones |
|---|---|---|
| 0 | Menú | 1 inicia una partida. Esc cierra el programa. |
| 1 | Partida | P abre la pausa. Q o Esc regresan al menú. Las reglas pueden activar la victoria o la derrota. |
| 2 | Victoria | R reinicia. M o Q regresan al menú. Esc cierra el programa. |
| 3 | Derrota | R reinicia. M o Q regresan al menú. Esc cierra el programa. |
| 4 | Pausa | P o Enter reanudan. Q o Esc regresan al menú. |

### Compilación y ejecución

Abrir **maze.asm** en EMU8086 4.08, con los tres archivos INC en la misma carpeta. Compilar como COM y guardar el ejecutable en **build/MAZE.COM**. En Windows, **JUGAR.bat** abre DOSBox-X con **build/play.conf** y monta la carpeta del juego.

## MAPEO DE MEMORIA

### Datos y pantalla

El programa COM comparte un segmento para código y datos. DS señala los datos del juego. ES se usa temporalmente para la copia del mapa y el acceso al video. SS:SP identifica la pila. En modo real, la dirección física se calcula como segmento × 16 + desplazamiento.

| Datos | Reserva | Offset en el COM | Uso |
|---|---|---|---|
| jugador_x, jugador_y | | | Columna y fila del jugador |
| vidas, monedas | 1 byte cada uno | 0105h, 0106h | Vidas y monedas pendientes. |
| puntos, record_max, movimientos | 2 bytes cada uno | 0107h, 0109h, 010Bh | Puntaje, récord de sesión y desplazamientos válidos. |
| estado | 1 byte | 010Dh | Estado de la aplicación. |
| mapa_original | 300 bytes | 010Eh | Diseño fijo del laberinto. |
| mapa | 300 bytes | 023Ah | Copia de trabajo donde se consumen monedas y trampas. |
| ui_number_buffer | 6 bytes | 06E1h | Cinco dígitos ASCII y el terminador de cadena. |

### Celdas y memoria de video

El mapa guarda 15 filas consecutivas de 20 celdas. El índice de una posición es **y × 20 + x**. La primera moneda, en (4, 1) con coordenadas desde cero, está en el índice 24. La carita del jugador se superpone al dibujar, mientras sus coordenadas se guardan aparte.

La pantalla de 80 × 25 usa 4000 bytes desde B800h. Cada posición contiene un carácter y un atributo de color. **ui_offset** calcula fila × 160 + columna × 2. El tablero comienza en la fila 6, columna 5, y cada celda lógica se dibuja con dos caracteres.

DESCRIPCIÓN DE MÓDULOS

## Responsabilidades y rutinas

**maze.asm · Inicio y estados**

**inicio** prepara los segmentos y el video. **mostrar_menu** recibe la selección y **nueva_partida** llama a **reiniciar_juego**. El ciclo de **dibujar_partida** espera teclado, delega el movimiento y elige qué pantalla mostrar. INT 21h con AX = 4C00h devuelve el control a DOS. **interfaz.inc · Pantallas y marcador**

**ui_init** selecciona el modo texto 03h mediante INT 10h. **ui_board** recorre el mapa y dibuja paredes,
monedas, trampas, salida y jugador. **ui_hud** presenta el marcador. **ui_number** obtiene los dígitos de derecha a izquierda mediante divisiones entre 10. **UI_LABEL** reutiliza la preparación de coordenadas, cadena y color para **ui_text**.

**movimiento.inc · Teclado y colisiones**

**procesar_movimiento** interpreta el código de las flechas en AH y las letras en AL.

**verificar_destino** comprueba los límites con comparaciones sin signo antes de indexar el mapa. Una pared cancela el intento. Un destino válido actualiza las coordenadas y el contador, y puede activar una moneda o trampa. La salida exige cero monedas pendientes.

**juego.inc · Reglas y reinicio sumar_moneda** agrega 10 puntos. **perder_vida** devuelve al jugador al inicio si quedan vidas y cambia a derrota cuando se agotan. **ganar_partida** agrega 50 puntos y activa victoria.

**actualizar_record** conserva el mayor puntaje. **reiniciar_juego** usa REP MOVSB para copiar 300 bytes y restaurar la partida, con el récord de la sesión intacto.

**Uso de pila y macros**

CALL guarda la dirección de retorno en SS:SP y RET la recupera. Las rutinas guardan los registros que necesitan preservar con PUSH y los recuperan en orden inverso con POP. Por ejemplo, **reiniciar_juego** guarda ES antes de copiar el mapa. Las macros DEFINE_INTERFACE, DEFINE_MOVIMIENTO y DEFINE_JUEGO integran los módulos en el ensamblado.

**BITÁCORA DE PRUEBAS**

**Compilación y primera partida**

La revisión corresponde a **MAZE.COM de 3376 bytes**. La figura E01 registra el ensamblado con EMU8086 4.08. Las capturas de partida corresponden a DOSBox-X 2026.10.01. Las acciones indican cómo reproducir cada caso. La última columna registra el resultado visible en las figuras conservadas en este informe.

| Figura | Acción para reproducir | Esperado | Observado |
|---|---|---|---|
| E01 | Compilar maze.asm con los tres INC. | Ensamblar y generar el COM. | Cuatro pasadas y un ejecutable de 3376 bytes. |
| E02 | Ejecutar MAZE.COM. | Ver el menú principal. | Menú visible con opciones 1 y Esc. |
| E03 | Pulsar 1 en el menú. | Iniciar en la celda inicial con los contadores base. | 3 vidas, 5 monedas, 0 puntos y 0 movimientos. |
| E04 | Desde el inicio, pulsar W hacia la pared. | Bloquear el desplazamiento. | Posición inicial y 0 movimientos al comparar con E03. |
| E05 | Pulsar D tres veces desde el inicio. | Recoger la primera moneda y sumar 10. | 10 puntos, 4 monedas pendientes y 3 movimientos. |
| E06 | Desde E05, salir de la celda y volver con A y D. | Conservar el puntaje. | 10 puntos y 4 monedas pendientes con 5 movimientos. |
| E07–E08 | Pulsar P, intentar mover y reanudar con Enter. | Pausar y continuar sin alterar la partida. | Pantalla de pausa y regreso a la misma celda de E06, con 10 puntos y 5 movimientos. |
| E09 | Volver al menú y comenzar otra partida después de puntuar. | Reiniciar el puntaje y conservar el récord. | 0 puntos, 10 de récord, 3 vidas y 5 monedas. |
| E10 | Desde el inicio, pulsar S, S, D, D. | Perder una vida y volver al inicio. | 2 vidas, retorno al inicio y desaparición de la primera trampa. |
| E11 | Repetir S, S, D, D. | Pasar por la celda sin perder otra vida. | El jugador llega a la celda y conserva 2 vidas. |

| BITÁCORA DE PRUEBAS | | Esperado | Observado |
|---|---|---|---|
| **Finales y ajustes de integración** | | Seguir en partida sin recibir la bonificación. | Jugador junto a la salida, 30 puntos y 2 monedas pendientes en ambas capturas. |
| **Figura** | **Acción para reproducir** | | |
| E12 | Intentar entrar a S con monedas pendientes. | | |
| E13 | Recoger las cinco monedas y en S. | Ganar con 100 puntos. | entrar Pantalla de victoria con 100 puntos y récord de 100. |
| E14 | Pulsar R tras ganar. | Restaurar la partida y conservar el récord. | 3 vidas, 5 monedas, 0 puntos, 0 movimientos y récord de 100. |
| E15 | Activar las tres trampas distintas. | Pasar al estado de derrota. | Pantalla de Game Over con el récord de 100 conservado. |
| E16 | Pulsar R tras perder. | Restaurar la partida sin salir del emulador. | Tablero inicial, 3 vidas, 5 monedas y récord de 100. |
| E17 | Intentar salir del tablero por un borde. | Permanecer dentro del laberinto. | La captura muestra al jugador dentro del mapa. Por sí sola no registra la tecla ni un antes y después. |

**Errores detectados durante la integración**

| Error | Impacto | Solución aplicada |
|---|---|---|
| Nombres distintos entre módulos | Interfaz y reglas no compartían los datos de la partida. | Unificar las variables compartidas de posición, vidas, puntaje y récord. |
| Símbolos de celda inconsistentes | El mapa dibujado no coincidía con la lógica. | Usar C para moneda, X para trampa y S para salida. |
| Ciclo de juego incompleto | El programa terminaba antes de procesar una partida. | Conectar el menú, el bucle de teclado y las transiciones de estado. |
| Inclusión de procedimientos en EMU8086 | El ensamblador no reconocía algunas rutinas incluidas. | Expandir los módulos con macros DEFINE y conservar archivos INC con CRLF. |

| | | |
|---|---|---|
| Ruta de montaje con caracteres acentuados | DOSBox-X no encontraba la mediante la carpeta del juego. | Montar la carpeta build ruta relativa de play.conf. |

### 3.1. Compilación del programa

El programa principal se ensambla en EMU8086 junto con los tres archivos de inclusión. La compilación produce MAZE.COM, el ejecutable utilizado en las pruebas. Esta evidencia vincula el código fuente con la versión que se pone en funcionamiento; el mensaje del ensamblador permite comprobar si el proceso finalizó sin errores.

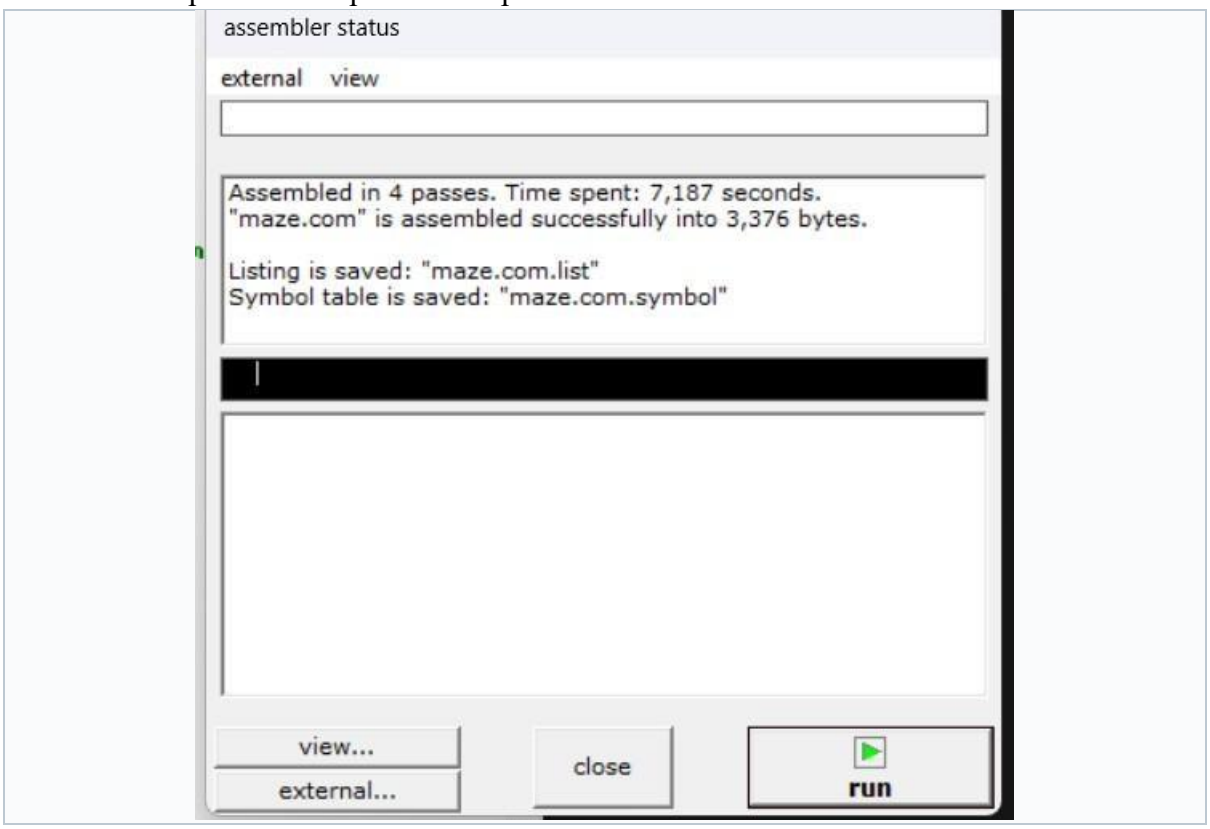



*Figura E01. Compilación del programa Maze8086 en EMU8086.*

### 3.2. Menú principal

Al ejecutar MAZE.COM en DOSBox-X aparece el menú principal, con el título del juego y las opciones disponibles. Esta pantalla es el punto de entrada de la aplicación: desde ella se puede comenzar una partida con la tecla 1 o salir del programa con Escape.




*Figura E02. Menú principal de Maze8086.*

### 3.3. Inicio de una partida

Al elegir la opción de inicio se restaura el mapa y se ubica al jugador en la posición inicial. El marcador parte con tres vidas, cinco monedas pendientes, cero puntos y cero movimientos. Estos valores permiten comparar las modificaciones que ocurren durante las demás pruebas.




*Figura E03. Tablero y marcador al comenzar una partida.*

### 3.4. Colisión con una pared

El desplazamiento se calcula primero como una posición candidata. La rutina de movimiento revisa el mapa antes de modificar las coordenadas del jugador; por eso, un intento de atravesar una pared no debe cambiar la posición ni aumentar el contador de movimientos. La captura se compara con el inicio de partida para demostrarlo.




*Figura E04. Intento de desplazamiento bloqueado por una pared.*

### 3.5. Recolección de una moneda

Al entrar en una celda con el símbolo C, la lógica del juego suma diez puntos y disminuye en una unidad las monedas pendientes. En la primera moneda, el marcador debe pasar a diez puntos y cuatro monedas. La posición del jugador y el marcador permiten relacionar el movimiento con la actualización de la puntuación.




*Figura E05. Actualización de la puntuación al recoger una moneda.*

### 3.6. Moneda recogida una sola vez

El mapa de la partida se modifica cuando se consume una moneda, reemplazando su símbolo por un espacio. Al salir de esa celda y regresar, el jugador puede atravesarla, pero la puntuación no debe aumentar otra vez. Esta comprobación evita que un mismo objeto produzca puntos repetidos.




*Figura E06. La moneda retirada del mapa no vuelve a generar puntos.*

3.7. Pausa de la partida

Durante el juego, la tecla P cambia el estado de la aplicación a pausa y se muestra una pantalla específica. La separación de estados impide que las reglas de movimiento se procesen mientras la partida está detenida, sin que sea necesario cerrar el programa ni perder los datos acumulados.

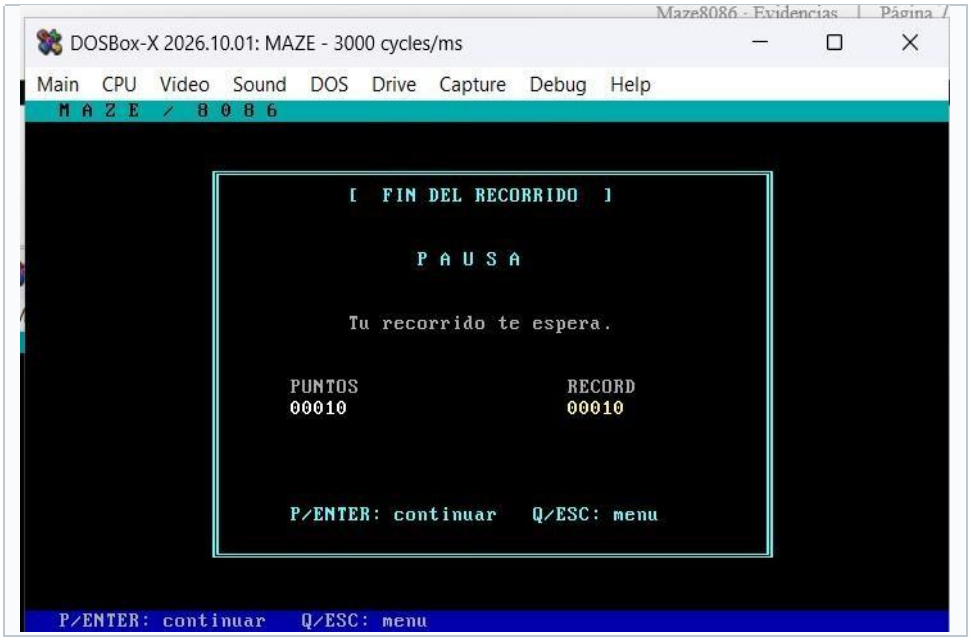



*Figura E07. Pantalla de pausa durante la partida.*

### 3.8. Continuación después de la pausa

En la pantalla de pausa, las teclas de desplazamiento no alteran la posición del jugador. Al pulsar Enter, la aplicación retorna al estado de partida con las mismas coordenadas, puntos y vidas. Esta evidencia se interpreta junto a la captura anterior para comprobar que la pausa no modifica el progreso.




*Figura E08. Continuación de la partida sin movimiento durante la pausa.*

3.9. Conservación del récord

Cuando se obtienen puntos, el juego compara la puntuación con el récord máximo y conserva el valor mayor. Si el jugador vuelve al menú y empieza otra partida, la puntuación vuelve a cero, pero el récord alcanzado debe permanecer visible. Este récord se guarda en memoria únicamente mientras MAZE.COM continúa abierto.




*Figura E09. Nueva partida con puntuación reiniciada y récord conservado.*

### 3.10. Pérdida de una vida por trampa

Las trampas están representadas por X. Cuando el jugador entra en una de ellas, la rutina correspondiente reduce una vida; si todavía quedan vidas, el personaje regresa a su posición inicial. En el primer contacto deben quedar dos vidas y el tablero mantiene el resto de las condiciones de la partida.




*Figura E10. Pérdida de una vida y regreso al inicio al tocar una trampa.*

3.11. Trampa consumida

Cada trampa debe afectar al jugador una sola vez. Después de activarse, la celda correspondiente se limpia en el mapa activo; por ello, regresar a ese lugar no ocasiona otra pérdida de vida. La comparación con la evidencia anterior permite identificar que el contador de vidas permanece sin cambios.




*Figura E11. Paso por una trampa previamente activada.*

### 3.12. Salida bloqueada con monedas pendientes

La salida está representada por S, pero no basta con llegar hasta ella: el juego verifica que el contador de monedas sea cero. Si aún queda alguna moneda, no se activa la victoria ni se asigna la bonificación final. Este control obliga a completar el recorrido antes de terminar la partida.

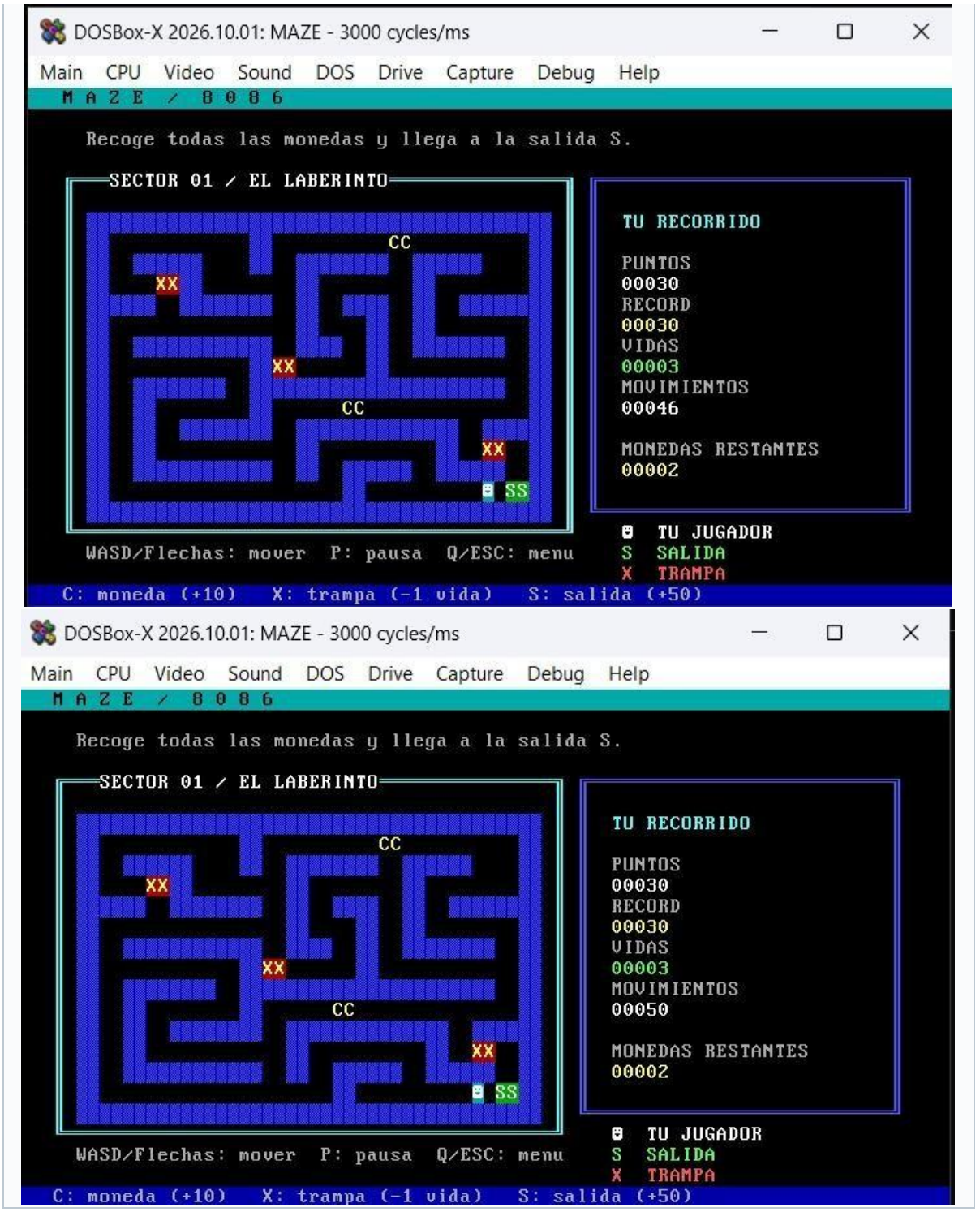



*Figura E12. Intento de salida con monedas pendientes.*

### 3.13. Condición de victoria

Después de recoger las cinco monedas, el marcador reúne cincuenta puntos por los objetos obtenidos. Al entrar en la salida, el juego agrega cincuenta puntos más, actualiza el récord cuando corresponde y muestra la pantalla de victoria. La puntuación final prevista es de cien puntos.




*Figura E13. Pantalla de victoria al completar el laberinto.*

### 3.14. Reinicio después de la victoria

La opción de reinicio restaura el mapa original y recupera las cinco monedas y las tres vidas, sin cerrar DOSBox-X. También reinicia la puntuación y los movimientos, mientras conserva el récord de la sesión. La copia del mapa permite que monedas y trampas vuelvan a sus posiciones iniciales.




*Figura E14. Partida restaurada después de una victoria.*

### 3.15. Condición de derrota

El jugador comienza con tres vidas y pierde una al entrar en cada trampa distinta. Cuando se agota la última vida, la aplicación cambia al estado de derrota y presenta la pantalla de Game Over. Se comprueba así una condición de finalización distinta de la victoria, vinculada al sistema de vidas.

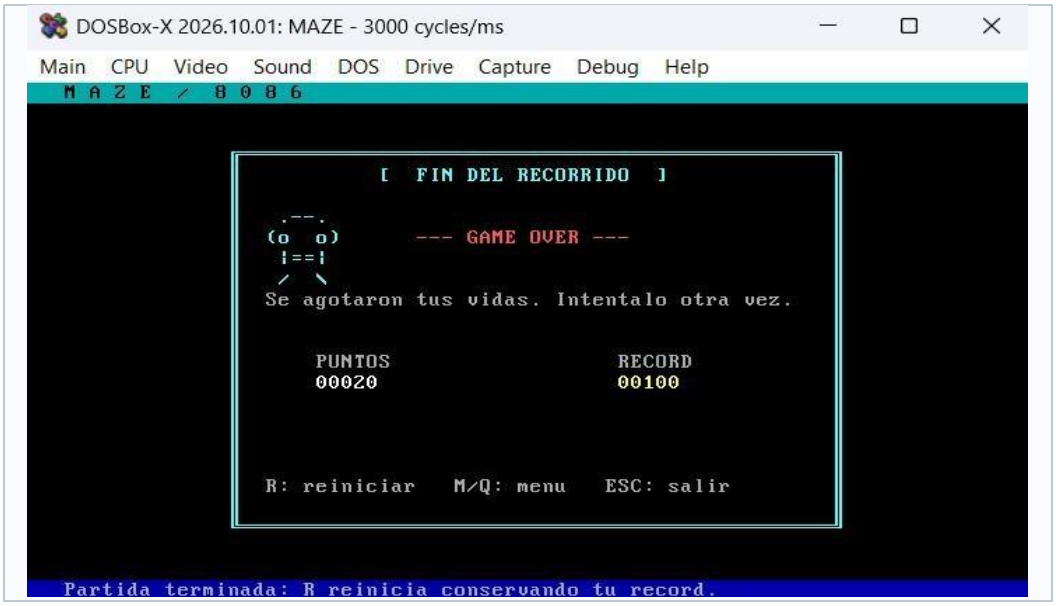



*Figura E15. Pantalla de derrota tras perder todas las vidas.*

### 3.16. Reinicio después de la derrota

Desde la pantalla de derrota también es posible comenzar una nueva partida con la tecla R. El mapa, las vidas, las monedas, el puntaje y los movimientos regresan a sus valores iniciales sin reiniciar el emulador. Esta función permite volver a jugar sin recargar el ejecutable.




*Figura E16. Nuevo inicio después de la derrota.*

## 4. Verificaciones complementarias

Además de las situaciones anteriores, el enunciado del proyecto destaca el control de límites, el historial de versiones y la explicación de las modificaciones realizadas. Estas evidencias sirven para respaldar esos aspectos durante la exposición y en el informe técnico.

### 4.1. Límites del laberinto

El mapa tiene veinte columnas y quince filas. Antes de consultar el contenido de una celda, la rutina de movimiento comprueba que las coordenadas candidatas estén dentro del rango permitido. La evidencia de esta prueba corresponde a intentos de desplazamiento en las zonas cercanas a los bordes: el jugador debe permanecer dentro del área del laberinto.




*Figura E17. Comprobación de los límites del mapa.*

### 4.2. Referencia original y versión modificada

El equipo utilizó como referencia el repositorio MICROPROCESOR-EMU8086ASSEMBLY-LANGUAGE. La comparación visual debe presentar el juego de referencia y Maze8086, identificando únicamente diferencias que puedan comprobarse en ambos programas. El documento de código del grupo describe un menú, marcador, monedas, trampas, pausa y estados de victoria y derrota; cualquier afirmación sobre qué se añadió o cambió debe confirmarse contrastando la versión original.




*Figura E18. Comparación entre el juego consultado como referencia y Maze8086.*

### 4.3. Organización del trabajo en Git

El enunciado requiere un repositorio Git con aportes distribuidos entre integrantes. El código del proyecto está dividido en archivos de interfaz, movimiento y reglas, pero esa estructura por sí sola no demuestra quién realizó cada cambio. La evidencia del repositorio debe mostrar el historial real de commits y, cuando sea posible, su relación con cada módulo.

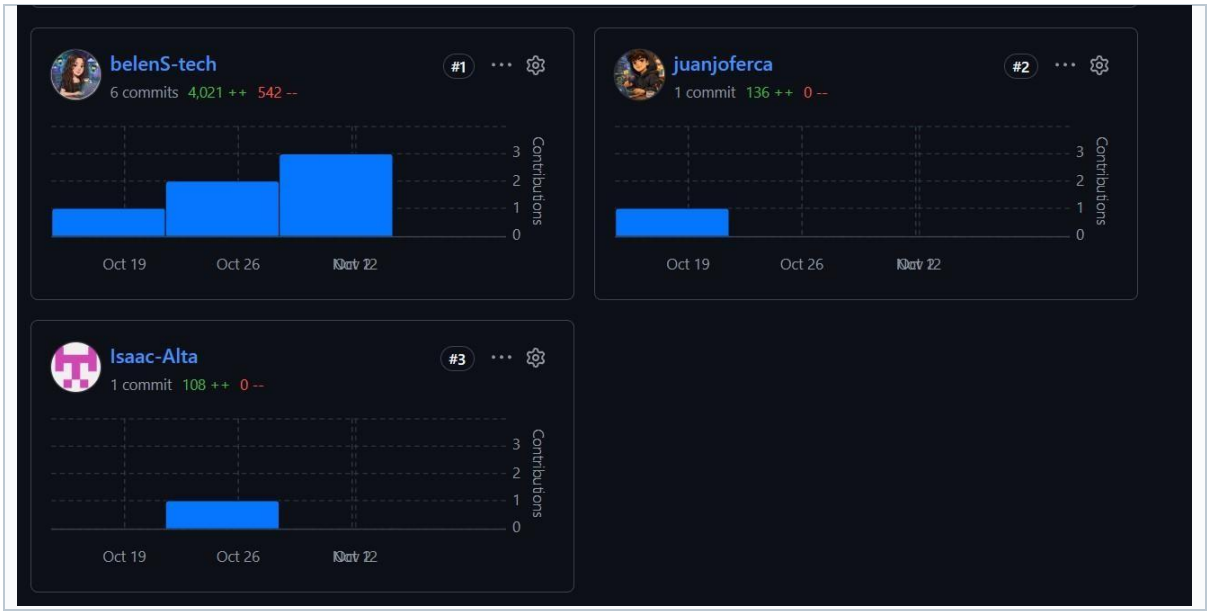



*Figura E19. Historial de versiones del repositorio del equipo.*

**Resultados y alcance**

Maze8086 aplica direccionamiento, registros, pila, procedimientos, macros e interrupciones a un laberinto en ensamblador 8086. La separación entre la matriz y el dibujo permite comprobar una celda antes de modificar la posición. El uso de estados mantiene diferenciados el menú, la partida, la pausa y los dos finales.

Las evidencias muestran que la compilación genera el ejecutable y que, al jugar en DOSBoxX, las paredes bloquean el paso, las monedas y trampas se consumen, y la salida exige recoger todas las monedas. El recorrido de victoria alcanza 100 puntos. Tras ganar o perder, el jugador puede reiniciar sin cerrar el emulador y conservar el récord de la sesión.

La restauración del tablero desde **mapa_original** permite recuperar los objetos y reiniciar los contadores. El juego tiene un solo mapa, un puntaje máximo de 100 y un récord que se conserva mientras el programa siga abierto.

**Aportes frente al juego de referencia**

La figura E18 muestra Escape Plan y Maze8086. El juego consultado usa un tablero numérico de 20 × 20 y las flechas. El equipo diseñó un tablero de 20 × 15 y añadió WASD, monedas, trampas consumibles, vidas, un marcador durante la partida y reinicio. La validación del equipo usa un índice común para las cuatro direcciones.

**Referencias**

Universidad Nacional. (2026). Proyecto 2: Proyecto de implementación de un juego en lenguaje ensamblador. Arquitectura de Computadoras (EIF205). Material del curso.

Equipo Maze8086. (2026). Código fuente de Maze8086: maze.asm, interfaz.inc, movimiento.inc y juego.inc, y archivos de documentación del proyecto.

Equipo Maze8086. (2026). *Maze8086-EIF205* [Repositorio de código]. GitHub. https://github.com/belenS-tech/Maze8086-EIF205

Ramaadarsh. (s. f.). MICROPROCESOR-EMU8086-ASSEMBLY-LANGUAGE [Repositorio de código]. GitHub. https://github.com/ramaadarsh/MICROPROCESOR-EMU8086-ASSEMBLY